TG Cloud 1.2.0 — Complete Technical & Forensic Architecture Report

Prepared from the supplied Android APK, extracted APK-forensics bundle, a real encrypted TG Cloud backup, a real encrypted multi-device sync node, a real 305-file `.link` share manifest, and the user's observed application behavior.

Date: 24 August 2026

Application: `com.telegram.cloud`

Version: `1.2.0`

---

0. Executive summary

TG Cloud is a Telegram-backed cloud-storage client rather than a conventional cloud-storage service with its own remote object-storage backend.

The Android build contains a complete application stack for:

- Telegram Bot API transport
- direct and chunked file upload/download
- multi-bot parallel transfer
- local encrypted database/state
- encrypted multi-device synchronization
- conflict detection/merging
- encrypted portable sharing manifests
- gallery synchronization/restoration
- chunked media streaming
- encrypted backup/restore
- Android background task scheduling
- a native C/C++ transfer/database/backup layer
- desktop database migration/interoperability code

The APK is unusually recoverable: first-party Kotlin/Java package names, class names, method names, fields, database schema strings and JNI entry points remain substantially intact. The native layer is harder to reconstruct because debug information is absent, but its exported JNI surface and many library/API symbols remain.

The most important architectural finding is that TG Cloud separates:

1. **physical Telegram storage** — individual Telegram documents/messages/chunks;
2. **logical cloud metadata** — `cloud_files`;
3. **synchronization state** — `sync_logs`, `sync_metadata`, encrypted sync nodes;
4. **portable sharing** — encrypted `.link` manifests;
5. **transfer state** — `upload_tasks` and `download_tasks`;
6. **media/gallery state** — `gallery_media`.

This separation explains how one Telegram channel can behave like a conventional cloud filesystem.

A real 252-chunk video record in the supplied backup contains exactly 252 Telegram message IDs and exactly 252 uploader-token assignments distributed over five configured bot identities. This directly supports the observed per-chunk multi-bot scheduling model.

A real encrypted sync node was decrypted and shown to contain one `INSERT` operation into `cloud_files`, with a `prevId` linking it into a synchronization chain. Therefore `sync_node_*.json.enc` is a sync-chain node, not simply a single

monolithic file index.

A real 305-file `.link` was decrypted. It contains 305 logical files (295 JPEG + 10 PNG), totaling 1,141,587,110 bytes, and is a portable encrypted manifest rather than the actual file data. It contains Telegram file/chunk identifiers and uploader information sufficient for the receiving application to retrieve the shared objects.

The application has explicit chunk-streaming classes (`StreamingChunkRequestBody`, `ChunkedStreamingManager`, `ChunkedVideoPlayer`) and offset-seeking support. This is consistent with the user's observation that large uploaded videos can be previewed/streamed without downloading the entire object first.

Important behavioral qualification: the APK contains methods named `resumeUpload`/`resumeChunkedUpload` and persisted `completed_chunks_json`, but the user's real-world behavior is that a failed upload is cancelled and must be manually started again from the beginning. Static presence of resume code must not be interpreted as proof that the current UI automatically resumes a cancelled network failure. The report therefore separates static code evidence from observed runtime behavior.

---

1. Evidence and confidence model

1.1 Primary artifacts

The investigation used:

- supplied APK: `com.telegram.cloud_1.2.0 (1).apk`
- APK forensic extraction bundle
- real backup: `tgcloud-backup-1787563792408.zip`
- real sync node: `sync_node_1786780783468.json.enc`
- real batch share file: `batch_305files-1gb.link`
- application screenshots supplied by the user
- user's observed behavior across multiple devices/files

1.2 APK hash

SHA-256:

`576cb4507401d6d0e2e0a242a49bee1069b55c06fa864868d9877db8bc18a559`

1.3 Confidence levels

Confirmed from static APK evidence: class/method/schema/build/JNI/crypto-format facts directly recovered from the supplied binary or extraction.

Confirmed from real artifact: a statement was tested against a real backup, sync node or `.link` supplied by the user.

Observed behavior: the statement comes from the user's actual use of the application.

Inference: architecture inferred from multiple confirmed facts. Inferences are explicitly labelled and should not be treated as source-code-level proof.

Unknown: the supplied artifacts do not establish the claim.

---

2. Application identity and build

2.1 Android identity

- Package namespace: `com.telegram.cloud`
- Version: `1.2.0`
- DEX size: 7,680,756 bytes

- First-party class definitions: 1,697
- First-party fields: 5,827
- First-party methods: 7,993

2.2 Toolchain

Recovered Kotlin/Gradle metadata indicates:

- Kotlin plugin: 1.9.24
- Gradle: 8.7
- Android Gradle Plugin: 8.5.0
- Android source/target compatibility: Java 17

2.3 Major Android dependencies

Recovered dependency metadata includes:

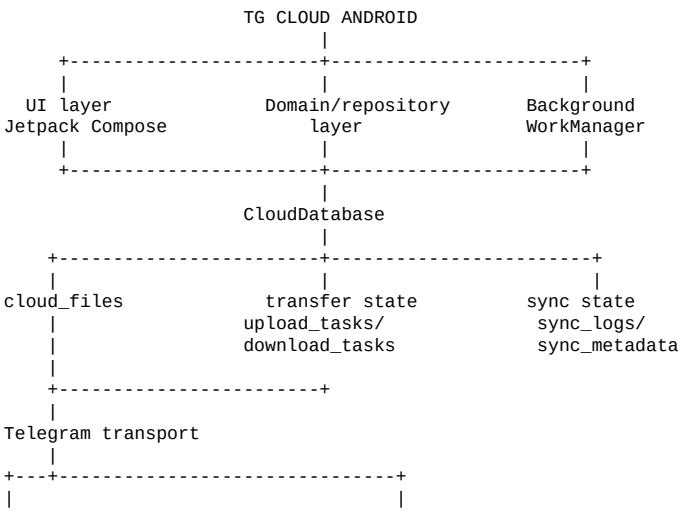
- Jetpack Compose 1.6.8
- Material 3 1.2.1
- AndroidX Activity 1.9.0
- AndroidX Lifecycle 2.8.2
- Navigation Compose 2.7.7
- Room 2.6.1
- AndroidX SQLite 2.4.0
- WorkManager 2.9.0
- DataStore 1.1.1
- Coroutines 1.8.1
- Material Components 1.12.0
- ExifInterface 1.3.7
- Media 1.7.0

This is a modern Kotlin/Android application using Compose for much of the UI and Room/SQLCipher-backed persistence.

---

3. High-level architecture

The recovered architecture can be represented as:



```
TelegramBotClient              multi-bot balancing
   |                                    |
   +----------------+------------------+
                    |
               Telegram Bot API
                    |
              Telegram channel
                    |
        +-----------+-------------+
        |                         |
 physical file/chunks       sync messages
        |                         |
   [CHUNK] objects           sync_node*.enc
```

Additional subsystems sit alongside the main storage path:

```
Sharing  -> encrypted .link manifests
Gallery  -> gallery_media + Telegram objects
Streaming -> chunk-aware media reads
Backup   -> encrypted ZIP containing DB/config
Native   -> JNI transfer/database/backup engine
```

---

4. Local data model

The application uses first-party Room-style tables backed by a SQLCipher-capable database.

Recovered tables:

- `cloud_files`
- `upload_tasks`
- `download_tasks`
- `gallery_media`
- `sync_logs`
- `sync_metadata`

Android/WorkManager also contributes standard scheduling tables such as `WorkSpec`, `WorkProgress`, `Dependency`, `WorkName`, `WorkTag`, and `SystemIdInfo`.

4.1 `cloud_files`

Recovered schema:

```
CREATE TABLE cloud_files (
    id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    telegram_message_id INTEGER NOT NULL,
    file_id TEXT NOT NULL,
    file_unique_id TEXT,
    file_name TEXT NOT NULL,
    mime_type TEXT,
    size_bytes INTEGER NOT NULL,
    uploaded_at INTEGER NOT NULL,
    caption TEXT,
    share_link TEXT,
    checksum TEXT,
    uploader_tokens TEXT DEFAULT ''
);
```

This table is the logical filesystem.

It contains both ordinary Telegram file identifiers and the metadata necessary to reconstruct chunked files.

Key conclusion

The Telegram channel is not itself the application's logical filesystem.

`cloud_files` is the local logical layer that maps a logical file to Telegram-backed storage.

---

5. Transfer-state database

5.1 `upload_tasks`

Recovered schema:

```
CREATE TABLE upload_tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    uri TEXT NOT NULL,
    display_name TEXT NOT NULL,
    size_bytes INTEGER NOT NULL,
    status TEXT NOT NULL,
    progress INTEGER NOT NULL,
    error TEXT,
    created_at INTEGER NOT NULL,
    file_id TEXT,
    completed_chunks_json TEXT,
    token_offset INTEGER NOT NULL
);
```

This is separate from `cloud_files`.

It therefore represents transfer state rather than the final logical cloud manifest.

Important fields:

- source URI
- display name
- byte size
- status
- progress
- error
- file ID
- completed chunk state
- token offset

5.2 `download_tasks`

Recovered schema:

```
CREATE TABLE download_tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
    file_id TEXT NOT NULL,
    target_path TEXT NOT NULL,
    status TEXT NOT NULL,
    progress INTEGER NOT NULL,
    error TEXT,
    created_at INTEGER NOT NULL,
    completed_chunks_json TEXT,
    chunk_file_ids TEXT,
    temp_chunk_dir TEXT,
    total_chunks INTEGER NOT NULL
);
```

This separately tracks chunked download reconstruction.

5.3 Important qualification about resume

Static APK evidence contains:

- `resumeUpload`
- `resumeDownload`
- `resumeChunkedUpload`
- `completed_chunks_json`

However, the user's actual current application behavior is:

1. upload starts;

2. network/light failure causes the upload to cancel;

3. user must manually select the file again;

4. second attempt starts from the beginning.

Therefore the correct conclusion is:

> **The binary contains transfer-resume machinery, but automatic resumability after the observed network-failure cancellation is not established and is contradicted by the user's current runtime experience.**

The resume methods may be used for another task state, explicit retry/recovery path, or an internal transfer workflow. This needs runtime tracing to determine.

---

6. Chunking system

6.1 Dedicated chunk manager

The APK contains:

- `ChunkedUploadManager`
- `ChunkedDownloadManager`
- `ChunkedUploadManagerKt.CHUNK_SIZE`
- `ChunkedUploadManagerKt.CHUNK_THRESHOLD`

The upload manager exposes:

- `uploadChunked`
- `resumeChunkedUpload`
- `uploadSingleChunk`

The exact method signatures include:

- source URI
- filename
- total size
- chunk lists
- token lists
- chunk index/state
- callbacks
- coroutine continuations

6.2 Physical representation

User screenshots show Telegram objects named in the form:

```
<logical-name>.mp4.chunk_<N>_of_<TOTAL>
```

and captions such as:

```
[CHUNK]
fileId:<logical-file-id>
chunk:<N>
total:<TOTAL>
name:<logical-name>
hash:<chunk-hash>
```

This is consistent with the database's chunk manifest representation.

6.3 Real 252-chunk evidence

The supplied backup contains a real file:

`Biological Classifications FULL CHAPTER | Class 11th Botany | Arjuna NEET [YXL0SxV1NFw].mp4`

Recovered:

- logical size: 1,053,618,346 bytes
- declared chunks: 252
- Telegram message IDs: 252
- uploader assignments: 252

The five configured uploader identities are distributed approximately evenly across the 252 chunks.

This is direct evidence that bot assignment is associated with individual chunks, not merely with entire files.

---

7. Multi-bot parallel transfer

The setup UI explicitly advertises:

- additional bot tokens
- faster uploads/downloads
- better handling of large files
- automatic load distribution
- up to 5x speed improvement

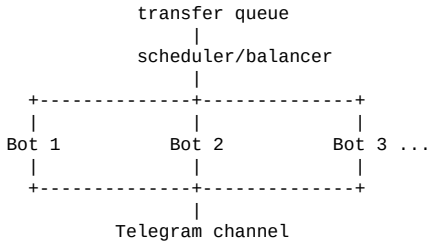
The Android code contains:

- `Balancer`
- `BalancerStats`
- `TokenRateLimiter`
- `TelegramBotClient`
- per-token transfer state
- token offset fields
- per-chunk uploader-token assignments

`TelegramBotClient` contains a token field and rate limiter.

The database stores `uploader_tokens` at the logical-file level and, for chunked manifests, preserves the per-chunk assignment.

Reconstructed scheduler model



For a chunked file:

```
chunk 0  -> bot 3
chunk 1  -> bot 1
chunk 2  -> bot 5
...
chunk 251 -> bot 2
```

The physical Telegram arrival order therefore does not need to match chunk order.

The application reconstructs logical order using chunk metadata.

---

8. Telegram transport layer

`TelegramBotClient` exposes methods including:

- `sendDocument`
- `sendChunk`
- `sendChunkStreaming`
- `getFile`
- `getMessageFileId`
- `downloadFile`
- `downloadFileToBytes`
- `forwardMessage`
- `sendTextMessage`
- `getPinnedMessage`
- `pinChatMessage`
- `deleteMessage`
- `getSyncMessages`
- `downloadSyncLog`

Recovered URLs include:

```
https://api.telegram.org/bot
https://api.telegram.org/file/bot
```

The application therefore uses Telegram's Bot API directly rather than merely embedding Telegram's normal client application.

The client has separate HTTP clients for ordinary and chunk transfers and a rate-limiting layer.

---

9. Rate limiting and scheduling

The APK contains:

- `TokenRateLimiter`
- rate-limit interval/window constants
- retry-after registration
- `BalancerStats`
- `TaskQueue`
- `TaskQueueManager`
- WorkManager workers
- progress notification management

This indicates the application was designed to deal with long-running asynchronous transfers rather than making a single synchronous API request.

---

10. Synchronization architecture

The synchronization subsystem contains:

- `SyncEngine`
- `SyncCrypto`
- `SyncLogManager`
- `SyncLogDao`

- `SyncMetadataDao`
- `SyncConfig`
- `ConflictResolver`
- `SyncOperation`
- `SyncOperationConverter`

10.1 `SyncConfig`

Recovered fields:

```
deviceId
syncBotToken
syncChannelId
syncPassword
isEnabled
```

The application therefore treats the sync configuration as distinct from ordinary storage configuration.

10.2 Synchronization journal

`sync_logs` schema:

```
CREATE TABLE sync_logs (
    log_id TEXT NOT NULL,
    timestamp INTEGER NOT NULL,
    device_id TEXT NOT NULL,
    operation TEXT NOT NULL,
    table_name TEXT NOT NULL,
    primary_key TEXT NOT NULL,
    data_json TEXT,
    previous_data_json TEXT,
    is_uploaded INTEGER NOT NULL,
    telegram_message_id INTEGER,
    checksum TEXT,
    PRIMARY KEY(log_id)
);
```

Operations are capable of representing record-level changes.

10.3 Synchronization checkpoints

`sync_metadata` contains key/value checkpoints with update timestamps.

The sync engine exposes:

- `performSync`
- `downloadNewLogs`
- `uploadPendingLogs`
- `applyRemoteLogs`
- `downloadSyncIndex`
- `fetchChainHead`
- `fetchNodeByForwarding`
- `uploadSyncNode`

This is a real synchronization protocol rather than simple periodic copying.

---

11. Sync-chain evidence from the real artifact

The supplied:

`sync_node_1786780783468.json.enc`

was successfully decrypted.

It contains:

- one sync entry
- `prevId = 18846`
- timestamp `1786780782614`
- operation `INSERT`
- table `cloud_files`
- primary key `2280`
- log ID `6d1f611d-fe4c-41b9-be53-8723f2c29897`

The timestamp corresponds to approximately:

`15 August 2026, 13:29:42.614 IST`

The entry's `dataJson` is a complete `cloud_files` record for the 252-chunk video.

Architectural conclusion

The object called:

```
sync_node_*.json.enc
```

is not itself the complete cloud index.

It is a node containing synchronization log entries.

The `prevId` field establishes a chain-like relationship between synchronization nodes.

---

12. Sync cryptography

`SyncCrypto` contains explicit constants and methods for:

- PBKDF2-based key derivation
- AES-GCM encryption/decryption
- Base64 encoding/decoding
- checksum generation/verification

Recovered format from the real node and APK:

```
PBKDF2-HMAC-SHA256
100,000 iterations
32-byte key
16-byte salt
12-byte GCM IV
128-bit authentication tag
AES-GCM
GZIP-compressed JSON plaintext
```

This is separate from both the backup and `.link` encryption formats.

---

13. Sync conflict resolution

`ConflictResolver` contains:

- `detectConflict`
- `resolveConflict`
- `tryMerge`
- `groupByRecord`
- `orderByTimestamp`
- modified-field extraction

Recovered conflict types include:

- `NONE`
- `DELETE_CONFLICT`
- `FIELD_CONFLICT`
- `IRRECONCILABLE`

Therefore multi-device synchronization is not just last-write-wins by necessity; the code contains explicit conflict classification and merge logic.

---

14. Why different phones can share a channel but have different clouds

Your observed experiment:

```
same channel
same bot
different sync password
```

produces different logical file views.

The APK independently establishes:

```
SyncConfig
 ■■■ syncChannelId
 ■■■ syncBotToken
 ■■■ syncPassword
```

and `SyncCrypto` derives encryption keys from the sync password.

Therefore a strong architecture explanation is:

```
physical storage identity
        !=
logical synchronized namespace
```

The channel/bot provides access to Telegram-backed objects, while the encrypted synchronization layer determines what logical state a device can reconstruct.

This is an architectural inference strongly supported by both code and the user's controlled experiment.

---

15. Sharing system

Sharing has a completely separate subsystem:

- `ShareLinkManager`
- `MultiLinkGenerator`
- `MultiLinkDownloadManager`
- `LinkDownloadManager`

The share layer supports:

- single-file link generation
- batch link generation
- dashboard selection
- gallery selection
- reading `.link` files
- downloading direct files
- downloading chunked files
- retrying chunk downloads

15.1 `.link` cryptography

Recovered implementation:

```
PBKDF2-HMAC-SHA256
10,000 iterations
32-byte derived key
16-byte salt
16-byte IV
AES-256-CBC
PKCS#5/PKCS#7-compatible padding
```

Binary layout:

```
16-byte salt
16-byte IV
AES-CBC ciphertext
```

The decrypted content is JSON and is not additionally compressed.

---

16. Real 305-file share artifact

The supplied:

`batch_305files-1gb.link`

decrypts successfully with the supplied password.

Recovered:

- version: `1.0`
- type: `batch`
- logical files: **305**
- total logical size: **1,141,587,110 bytes**
- 295 JPEG
- 10 PNG
- 196 direct files
- 109 chunked files
- 108 two-chunk files
- 1 four-chunk file
- 220 physical chunk records

The manifest contains fields for:

- file name
- size
- MIME type
- category
- upload date
- Telegram file ID
- uploader identity
- encryption state
- chunk number
- total chunks
- chunk size
- chunk hash

- Telegram chunk file ID

Conclusion

The `.link` is a portable encrypted manifest.

It is not a copy of the actual shared data.

The recipient can use the manifest to locate the Telegram-backed objects and add them to the recipient's existing logical cloud.

---

17. Security implication of `.link`

Because the manifest contains Telegram access information, possession of a decrypted share manifest can expose sensitive transport credentials.

The password used in the supplied test artifact was intentionally simple.

This demonstrates a critical security property:

> The security of the share manifest is strongly dependent on the strength and secrecy of the share password.

The report intentionally does not reproduce any actual bot tokens or passwords.

Any bot token exposed in screenshots or files should be revoked and regenerated.

---

18. Backup architecture

`BackupManager` exposes:

- `createBackup`
- `restoreBackup`
- encrypted backup import
- desktop database migration
- Android migration
- SQLCipher migration
- ZIP/unzip operations
- `.env` parsing/building

The real backup contained:

```
telegram_cloud.db.enc
.env.enc
backup_manifest.json
```

It does not contain the actual multi-gigabyte Telegram-hosted file contents.

It is therefore a metadata/configuration backup.

---

19. Backup encryption

The backup wrapper format was recovered from the APK's `decryptFile` implementation and verified against the real backup.

Format:

```
BKP1
16-byte salt
16-byte IV
AES-256-CBC ciphertext
```

Key:

```
SHA-256(UTF-8(password) || salt)
```

Padding:

```
PKCS#7
```

This is deliberately distinct from the sync and share cryptographic formats.

---

20. Real backup database

The supplied backup decrypts to a database containing:

Table	Rows
`cloud_files`	2,168
`sync_logs`	2,177
`gallery_media`	786
`upload_tasks`	33
`download_tasks`	29
`sync_metadata`	4

The backup therefore provides a real snapshot of the application's local logical state.

---

21. Gallery subsystem

Recovered components:

- `GallerySyncManager`
- `GalleryRestoreManager`
- `GalleryMediaDao`
- `GalleryMediaEntity`
- `MultiFileGalleryManager`
- `MediaScanner`
- `ThumbnailCache`

`gallery_media` stores:

```
local_path
filename
mime_type
size_bytes
date_taken
date_modified
width
height
duration_ms
thumbnail_path
is_synced
telegram_file_id
telegram_message_id
telegram_file_unique_id
telegram_uploader_tokens
sync_error
last_sync_attempt
deleted_at
```

The gallery therefore has its own metadata layer rather than simply querying `cloud_files`.

`GallerySyncManager` is connected to both the chunked upload/download managers and the synchronization log manager.

---

22. Gallery parallelism

`GalleryRestoreManagerKt` contains a `MAX_PARALLEL_DOWNLOADS` constant.

`GallerySyncManagerKt` contains a `MAX_PARALLEL_UPLOADS` constant.

Therefore gallery synchronization is independently parallelized.

---

23. Media streaming

Recovered classes:

- `ChunkedStreamingManager`
- `ChunkedVideoPlayer`
- `StreamingChunkRequestBody`

`StreamingChunkRequestBody` includes an explicit `skipToOffset` operation.

This is strong evidence for offset-based reads rather than requiring a complete file download before playback.

The Telegram client also has:

```
sendChunkStreaming
```

and download methods capable of writing to streams/output streams.

Likely streaming flow

```
video player requests byte/range position
          |
          v
ChunkedStreamingManager
          |
          v
identify required logical chunk(s)
          |
          v
Telegram getFile/download
          |
          v
StreamingChunkRequestBody
          |
          v
seek/skip to required offset
          |
          v
media decoder
```

The exact runtime range-selection algorithm has not been fully reconstructed, so this diagram is architectural rather than a claim about every internal branch.

---

24. Image/video preview behavior

Your observed behavior is:

- JPEG/PNG can be previewed from cloud;
- videos can be streamed/previewed;
- arbitrary file formats do not necessarily have an equivalent inline preview.

The APK contains dedicated gallery, thumbnail and video-streaming subsystems, which explains why media types receive special treatment.

The absence of a universal preview path for arbitrary binary formats is consistent with the recovered architecture.

---

25. Large-file behavior

The APK explicitly contains:

- chunk thresholds
- chunked upload/download managers

- chunked streaming
- multi-token balancing
- persisted chunk state
- Telegram file identifiers
- per-chunk hashes
- large-file gallery handling

The user has successfully uploaded files in the multi-gigabyte range, including a 5.3 GB video that streamed successfully.

No authoritative hardcoded 10 GB maximum has been established.

Therefore:

> **10 GB is a user-observed successful upload scale, not a proven application maximum.**

The report does not claim a 10 GB limit.

---

26. Interrupted-upload / playback anomaly

Observed by the user:

- an upload can fail/cancel when connectivity disappears;
- the user must manually start the file again;
- at least one approximately 2 GB video that had a prior failed attempt later uploaded successfully but did not stream;
- a 5.3 GB video was successfully streamed and the user remembers that it also had a previous failed upload.

With only a small number of observations, this does **not** establish a causal bug.

Possible explanations include:

1. file/container/codec-specific behavior;
2. stale/orphaned metadata after a failed attempt;
3. a streaming-manifest/finalization issue;
4. chunk mapping inconsistency;
5. unrelated file-specific media properties.

The 5.3 GB successful playback makes a simple "videos over 2 GB cannot stream" explanation unsupported.

A controlled same-file experiment is required before attributing the issue to failed-upload history.

---

27. Native architecture

The APK contains:

```
libtelegramcloud_core.so
```

for:

- ARM64
- ARMv7

The native library exports 12 JNI entry points:

```
nativeInit()
nativeOpenDatabase(...)
nativeCloseDatabase()
nativeExportBackup(...)
nativeImportBackup(...)
nativeImportEncryptedBackup(...)
nativeStartDownload(...)
nativeStartTransfer(...)
nativeCancelTransfer(...)
```

```
nativeStopDownload(...)
nativeStartUpload(...)
nativeGetDownloadStatus(...)
```

The ARM64 ELF is AArch64 and is dynamically linked against Android system libraries.

Compiler/toolchain evidence:

- Android Clang 14.0.7
- LLD 14.0.7

No DWARF debug sections were found.

---

28. Native libraries and capabilities

Native strings/symbols indicate use of:

- libcurl
- MIME/form APIs
- OpenSSL
- SQLite/SQLCipher
- SHA-256
- MD5
- filesystem operations
- JSON transfer parsing
- backup import/export
- progress callbacks
- completion/failure callbacks

The native layer therefore appears to provide a lower-level transfer/database/backup engine behind the Kotlin/Java layer.

---

29. JNI architecture

The native library is not simply a cryptographic helper.

It exposes APIs for:

- database opening/closing
- uploads
- downloads
- transfer cancellation
- transfer status
- backup export
- backup import
- encrypted backup import

This makes the native layer a substantial application subsystem.

---

30. Desktop interoperability

The Android `BackupManager` contains methods explicitly named:

- `migrateDesktopDatabase`
- `migrateFromDesktopToAndroid`
- `migrateFromSQLCipher`

The native strings also contain:

- `TelegramCloudBackup`
- `.env`
- manifest handling
- `BOT_TOKEN`
- `CHANNEL_ID`

This is strong evidence that the Android application was designed to understand a desktop-side data/configuration format.

It does not, by itself, prove that the deleted desktop repository can be reconstructed byte-for-byte.

However, it gives a strong fingerprint for locating or rebuilding the desktop application.

---

31. UI architecture

Recovered UI is Jetpack Compose based and includes screens/components for:

- dashboard
- setup wizard
- bot configuration
- channel configuration
- multi-device sync
- multi-token boost
- gallery
- task queue
- themes
- transitions
- media viewing

The supplied screenshots align closely with the recovered class/resource structure.

The setup wizard requires:

1. Telegram bot token
2. private Telegram channel
3. optional multi-device sync
4. optional additional bot tokens
5. verification

The channel UI explicitly requires the bot to have administrator rights.

---

32. Storage identity vs transport identity

One of the strongest architectural conclusions is:

```
Bot token
    !=
logical user/storage identity
```

A bot token is used as a Telegram transport credential.

The logical cloud state is represented separately by:

- `cloud_files`
- synchronization records
- encrypted sync configuration/password
- device identity

This explains how multiple devices/users can share the same physical Telegram channel while maintaining distinct logical synchronization namespaces when their sync credentials differ.

This is an inference supported by:

- `SyncConfig`
- `SyncCrypto`
- the real sync node
- the database schema
- the user's same-channel/same-bot/different-password experiment.

---

33. Physical order vs logical order

Telegram message arrival order does not need to equal file chunk order.

A chunked file has:

```
logical file
   |
   +-- chunk 0
   +-- chunk 1
   +-- ...
   +-- chunk N
```

while Telegram may receive:

```
chunk 17
chunk 2
chunk 18
chunk 4
chunk 0
...
```

The application's per-chunk metadata and message/file identifiers provide the information needed to reconstruct logical order.

The real 252-chunk database record demonstrates that all 252 Telegram message IDs and 252 uploader assignments are stored in the logical manifest.

---

34. Why the application can look like "cloud storage"

The illusion of a normal cloud filesystem is produced by layering:

```
Telegram messages
      ↓
chunk/object manifest
      ↓
cloud_files logical records
      ↓
sync journal
      ↓
local database
      ↓
Compose file browser
```

Telegram supplies durable remote objects and transport.

TG Cloud supplies:

- naming
- hierarchy/listing
- chunk mapping
- synchronization
- encryption
- parallel scheduling
- sharing
- media streaming
- backup/restore
- UI
- local state

This is the core design.

---

35. Architectural strengths

Based on the recovered design:

Strong points

- no dedicated VPS is required for the storage backend;
- Telegram provides remote object transport/storage;
- multiple bots increase parallel transfer capacity;
- per-chunk bot assignment permits fine-grained load distribution;
- logical metadata is separated from physical Telegram objects;
- sync is journal-based rather than full-database replication;
- sync nodes are encrypted/authenticated;
- sharing is a portable encrypted manifest;
- gallery has dedicated sync/restore support;
- media has dedicated streaming code;
- backup contains portable local state rather than requiring re-indexing everything;
- desktop migration hooks exist;
- transfer state is persisted locally;
- native code provides lower-level transfer/database/backup operations.

---

36. Architectural weaknesses / risks

36.1 Bot credentials in share manifests

The decrypted `.link` format contains uploader credentials.

This creates a high-value secret-bearing artifact.

36.2 Weak share password

A short/simple share password provides inadequate protection for such a manifest.

36.3 Backup credential exposure

The backup contains encrypted configuration including bot credentials. A compromised backup password can therefore expose transport configuration.

36.4 Orphaned chunks

If an upload fails after some chunks are posted, Telegram may contain physical objects that are no longer referenced by a completed logical record.

The exact cleanup behavior was not established.

36.5 Streaming/finalization edge cases

The user's failed-upload playback observations suggest a possible edge case, but this remains unproven.

36.6 Native code complexity

The native layer is substantially harder to audit/reconstruct than the Kotlin layer because source-level debug information is absent.

36.7 Telegram dependency

The storage system is fundamentally dependent on Telegram Bot API availability, limits, rate behavior and message/file semantics.

---

37. What has been proven vs not proven

Proven

- package/version
- build/toolchain fingerprints
- 1,697 app classes
- 5,827 fields
- 7,993 methods
- native ARM64/ARMv7 library
- 12 JNI entry points
- Telegram Bot API usage
- chunked transfer subsystem
- multi-token/balancer subsystem
- local database schema
- sync subsystem
- conflict resolver
- gallery subsystem
- streaming subsystem
- backup subsystem
- desktop migration methods
- exact backup encryption wrapper
- exact `.link` encryption format
- exact sync-node encryption format
- real 2,168-file backup snapshot
- real 252-chunk logical file
- real 305-file share manifest

- real one-entry sync node

Strong inference

- channel/bot is physical transport while sync password defines an encrypted logical namespace;
- sync nodes form a chain;
- Telegram message order is not logical chunk order;
- `.link` is a portable retrieval manifest;
- multi-bot allocation occurs at chunk level.

Not proven

- a 10 GB hard maximum;
- automatic resumability after network failure;
- exact streaming algorithm for every media format;
- exact orphaned-chunk cleanup behavior;
- exact desktop application's original source;
- exact reason for the user's failed-upload playback anomaly.

---

38. Reverse-engineering recoverability

The Kotlin/Java layer is unusually recoverable because names such as:

```
ChunkedUploadManager
TelegramBotClient
SyncEngine
SyncCrypto
ShareLinkManager
BackupManager
GallerySyncManager
ChunkedStreamingManager
ConflictResolver
```

survive in the APK.

The database schema is also recoverable from embedded SQL strings.

Therefore a clean-room reimplementation can reproduce the architecture substantially more easily than an ordinary heavily obfuscated Android APK.

The native layer is harder but still tractable because:

- JNI names survive;
- exported symbols survive;
- library/API references survive;
- strings survive;
- crypto/library calls survive;
- transfer JSON parsing references survive.

It will not be possible to claim that a reconstructed native implementation is identical to the author's original source without the original repository.

---

39. Most important reconstructed data flows

Upload

```
User selects file
```

```
       ↓
UploadRequest
       ↓
size/threshold decision
       ↓
direct upload OR chunked upload
       ↓
TaskQueue / WorkManager
       ↓
Balancer
       ↓
bot selection
       ↓
TelegramBotClient
       ↓
Telegram channel
       ↓
physical Telegram object(s)
       ↓
logical cloud_files record
       ↓
sync log
       ↓
sync node
```

Chunked upload

```
file
 ↓
chunk N
 ↓
select bot
 ↓
sendChunk()
 ↓
Telegram message/document
 ↓
record Telegram IDs + hash + bot assignment
 ↓
repeat
 ↓
final logical manifest
```

Sync

```
local DB mutation
 ↓
sync_logs
 ↓
pending logs
 ↓
SyncCrypto
 ↓
encrypted sync node
 ↓
Telegram sync channel
 ↓
chain head
 ↓
remote device fetches nodes
 ↓
decrypt
 ↓
ConflictResolver
 ↓
applyLog()
 ↓
cloud_files
```

Share

```
selected files
 ↓
ShareLinkManager
 ↓
collect Telegram IDs/chunk manifests
 ↓
JSON
 ↓
PBKDF2
 ↓
AES-CBC
 ↓
.link
```

Recipient:

```
.link
 ↓
password
 ↓
decrypt
 ↓
manifest
 ↓
direct/chunked download
 ↓
Telegram
 ↓
local file
 ↓
optionally add to logical cloud
```

Backup

```
SQLite/config
 ↓
ZIP
 ↓
BKP1 encryption
 ↓
backup ZIP
```

Restore:

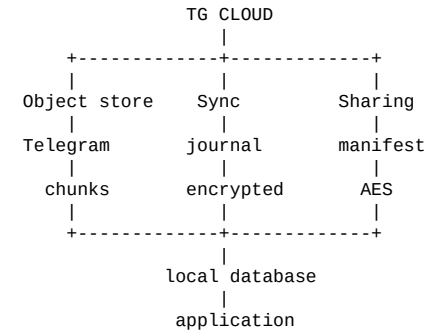
```
backup
 ↓
decrypt
 ↓
unzip
 ↓
restore DB/config
 ↓
migration if required
 ↓
cloud client resumes from restored state
```

---

40. Final technical assessment

TG Cloud is not merely a Telegram file uploader with a pretty frontend.

The recovered APK implements a fairly complete storage abstraction:



The cleverest part is the separation between **physical storage** and **logical state**.

Telegram provides the raw remote objects. TG Cloud supplies the metadata layer that turns those objects into files, the synchronization protocol that replicates that metadata, the multi-bot scheduler that increases throughput, and the streaming layer that allows media to be consumed without reconstructing the entire file first.

The real artifacts make this more than a theoretical reconstruction:

- a real 2,168-record database snapshot;
- a real 252-chunk video manifest;
- a real encrypted sync-chain node;
- a real 305-file encrypted share manifest.

Together they expose most of the core data model.

The remaining major unknowns are primarily **runtime edge cases and exact implementation details**, especially the streaming/finalization path after failed uploads and the complete native transfer engine.

---

41. Recommended next forensic targets

If the goal is to recreate the application rather than merely document it, the highest-value next targets are:

1. **Decompile** `SyncEngine` **completely**

- establish exact chain-head/index semantics;

- reconstruct node creation and traversal.

2. **Trace** `ShareLinkManager`

- reconstruct exact `.link` schema;

- understand import/add-to-cloud semantics.

3. **Trace** `ChunkedUploadManager.uploadSingleChunk`

- establish exact chunk naming/caption/hash format;

- identify finalization behavior.

4. **Trace** `ChunkedStreamingManager` **+** `ChunkedVideoPlayer`

- determine how byte offsets map to chunks;

- identify why some completed uploads may fail to stream.

5. **Trace** `BackupManager`

- reconstruct desktop migration schema;

- identify the missing desktop application's database format.

6. **Reverse the native JNI transfer engine**

- reconstruct `nativeStartTransfer`;

- identify its JSON request schema;

- map callbacks to Kotlin task state.

7. **Recover the exact five-token scheduler**

- determine whether allocation is round-robin, load-based, availability-based, or queue-based;

- correlate `token_offset`, uploader assignments and balancer state.

8. **Search globally for desktop fingerprints**

- class names;

- schema names;

- backup identifiers;

- `.env` keys;

- `TelegramCloudBackup`;

- sync-node filenames;

- share manifest field names.

---

42. Artifact inventory

Primary APK:

`com.telegram.cloud_1.2.0 (1).apk`

APK SHA-256:

`576cb4507401d6d0e2e0a242a49bee1069b55c06fa864868d9877db8bc18a559`

Native ARM64:

`libtelegramcloud_core.so`

ARM64 SHA-256:

`e668853d1ccd07513d778fd14eceb48ceea28a98302bc8b94c2a822c9d19cec7`

Native ARMv7:

`libtelegramcloud_core.so`

ARMv7 SHA-256:

`4c78ecaebc2ad888c03bbf1d9911ca2b03e1a0677890e79cc72e7aa2dfd13288`

---

43. Bottom line

The project is technically far more sophisticated than its ~33 MB APK size suggests.

Its core trick is not "store files in Telegram."

Its core trick is:

> **Turn Telegram's message/file infrastructure into a distributed object store, then build a logical filesystem, synchronization journal, encrypted portable manifests, multi-bot scheduler, gallery layer and chunk-aware media streaming system on top of it.**

That architecture is what makes the application behave like cloud storage without operating a conventional storage backend.

This report intentionally separates verified APK/artifact facts from inference and from user observations.